Capítulo 6. Explorar y modificar sistemas desconocidos

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *Parte de la experiencia más valiosa que adquirí fue al dar soporte a una aplicación heredada. Lo recomiendo encarecidamente, pero no se lo deseo a nadie.*
>
> —Dalia Abo Sheasha, desarrolladora de software

Aunque a muchos desarrolladores les encanta el lienzo en blanco de los proyectos nuevos, la cruda realidad del desarrollo de software es que la mayor parte del trabajo que hacemos se realiza en sistemas ya establecidos. No dejes que esta realidad desaliente tu visión de la profesión de desarrollador de software. Trabajar con bases de código existentes tiene sus ventajas. Muchas decisiones clave ya se han tomado, lo que te permite centrarte directamente en los problemas que se te presentan en lugar de preocuparte por la infraestructura.

Como persona que se incorpora a un nuevo proyecto o que simplemente explora un nuevo código base, necesitarás desarrollar habilidades para explorar y modificar sistemas desconocidos. En este capítulo, descubrirás cómo navegar y comprender código desconocido y cómo realizar cambios de forma segura. Tanto si te incorporas a un nuevo equipo como si mantienes sistemas heredados, estas habilidades te ayudarán a contribuir con confianza a cualquier código base. Esto se basa en la mecánica de lectura de código que hemos visto en el capítulo 2.

Comprender bases de código desconocidas

Tanto si te incorporas como desarrollador de software en una nueva empresa como si pasas a un nuevo proyecto en tu actual lugar de trabajo, trabajar con bases de código desconocidas puede resultar estresante y provocar ansiedad. Esto puede deberse a lo desconocido o a trabajar con lenguajes, marcos o herramientas con los que no estás familiarizado. Sin embargo, si abordas este reto con un plan claro y técnicas para navegar por estas aguas desconocidas, aumentarás tanto tu tasa de éxito como tu confianza.

En esta sección, aprenderás técnicas para comprender el panorama general o los objetivos del proyecto. Aprenderás a familiarizarte con un nuevo proyecto siguiendo el flujo o la ruta de ejecución del código. Por último, aprenderás a construir modelos mentales de forma incremental, dividiendo los sistemas complejos en partes manejables. A medida

que avanzas en este capítulo o exploras un nuevo proyecto, recuerda que no pasa nada si no entiendes inmediatamente todo lo que hay en una nueva base de código. Ni siquiera los miembros del equipo que llevan años trabajando en el proyecto lo saben todo sobre él.

Empieza con una visión general

«Los árboles no te dejan ver el bosque». Esto significa que si te centras demasiado en los pequeños detalles (qué versión del marco de trabajo estás utilizando), no logras comprender el panorama general o la situación global (el objetivo de este proyecto). Probablemente quieras sumergirte y empezar a contribuir al proyecto y demostrar tu valía al equipo de inmediato. Siempre es bueno tener un plan, y aquí tienes algunos consejos para comprender el panorama general.

Comprender el proyecto

Antes de empezar a comprender el código, debes comprender el propósito y la intención del proyecto. Empieza por lo general: ¿cuál es la razón general por la que existe este proyecto? ¿Quiénes son las partes interesadas en este proyecto? ¿Qué significa este proyecto para tu empleador? ¿Es una parte importante del negocio?

Hay muchas formas de hacerlo, pero un buen punto de partida es alguien que tenga un profundo conocimiento del producto, como un gestor de producto. Intenta concertar una reunión con esta persona y prepárate para aprender. Toma tantas notas como puedas para poder consultarlas más adelante. Si se trata de una reunión virtual, pregunta si

puedes grabarla para poder revisarla más tarde. Y lo más importante, haz preguntas. Las únicas preguntas tontas son las que no se hacen.

Después de hablar con el propietario del producto y obtener una visión general del proyecto, es el momento de profundizar en el aspecto técnico. En este punto, ponte en contacto con los ingenieros de software que trabajan actualmente en el proyecto y, si encuentras a alguno que haya desarrollado originalmente el sistema, considéralo como si hubieras encontrado oro durante la fiebre del oro. Lo ideal es que te pongas en contacto con alguien que ocupe un puesto de responsable técnico o de arquitectura y que pueda proporcionarte información valiosa sobre las decisiones que se tomaron.

¿Quiénes son los clientes de este producto? Busca cualquier información pública sobre este proyecto. Si la hay, intenta analizarla desde la perspectiva de los clientes. Algunas empresas desarrollan perfiles de usuario para sus productos, que son perfiles ficticios de los usuarios reales que pueden ayudarte a comprender los diferentes tipos de usuarios, sus objetivos y sus puntos débiles. Esto te proporcionará un contexto valioso para entender por qué se tomaron determinadas decisiones técnicas.

Revisar la documentación disponible

Después de hablar con las partes interesadas, el siguiente paso para comprender el código base es revisar toda la documentación disponible, incluidos los ADR ( ). Con una comprensión general del proyecto en sí, parte de esta documentación debería empezar a tener sentido.

En este punto, estás tratando de aprender todo lo que puedas sobre el código base a través de la documentación.

Una parte importante de esto es la famosa «documentación de incorporación» a un proyecto. Esta te dirá todo lo que necesitas saber para poner en marcha este proyecto en tu máquina local. Esta documentación ha mejorado a lo largo de los años con herramientas como Docker, pero su revisión sigue siendo un proceso tedioso.

Aunque tu trabajo no es escribir documentación en este momento, ya que todavía te estás familiarizando con todo, deberías tomar notas e identificar las lagunas en la documentación. Esto es algo a lo que podrás volver más adelante para corregirlo y es una forma estupenda de contribuir al equipo.

LA TRAMPA DE LA DOCUMENTACIÓN

Aquí Dan,. Una vez pasé tres días tratando de comprender un complejo sistema de autenticación, leyendo atentamente lo que creía que era documentación actualizada. Cuando finalmente le pedí ayuda a un compañero de equipo, se rió y me dijo: «Oh, esa documentación es de hace dos versiones. El año pasado reescribimos completamente el sistema de autenticación». ¿La lección? La documentación puede mentir, pero el código nunca lo hace. Verifica siempre lo que lees con lo que realmente se está ejecutando. Y no tengas miedo de pedir ayuda. Una conversación de tres minutos me habría ahorrado tres días.

Comprender la arquitectura y la estructura del proyecto

Después de comprender el proyecto en general y leer toda la documentación disponible, es hora de examinar la arquitectura y la organización del código base. Este es un paso importante antes de sumergirte en la lectura de cualquier código o configuración, o en la escritura de código. La estructura del proyecto puede revelar mucha información y darte una visión más holística del mismo, lo que te ayudará a navegar por el código base.

La mayoría de los proyectos siguen algún patrón arquitectónico e, ya sea por diseño deliberado o por evolución orgánica. Empieza por identificar qué patrón (o combinación de patrones) sigue el código base.

E*l empaquetado por capas* organiza el código horizontalmente en función de las responsabilidades técnicas. A menudo verás directorios de nivel superior como controladores, servicios, repositorios y modelos. Este enfoque agrupa componentes técnicos similares, lo que facilita la búsqueda de todos los componentes de un tipo concreto. El siguiente código está empaquetado por capas:

```
src/
  ├── controllers/
  │   ├── UserController.java
  │   └── ProductController.java
  ├── services/
  │   ├── UserService.java
  │   └── ProductService.java
  ├── repositories/
  │   ├── UserRepository.java
```

```
    │   └── ProductRepository.java
    └── models/
        ├── User.java
        └── Product.java
```

El empaquetado por características organiza el código verticalmente en torno a las capacidades o características empresariales. Verás directorios de nivel superior que representan dominios empresariales como usuarios, productos y pedidos. Este enfoque encapsula todos los aspectos de una característica, lo que facilita la comprensión de los flujos de trabajo empresariales completos. El siguiente es un ejemplo de código empaquetado por características:

```
src/
  ├── users/
  │   ├── UserController.java
  │   ├── UserService.java
  │   ├── UserRepository.java
  │   └── User.java
  └── products/
      ├── ProductController.java
      ├── ProductService.java
      ├── ProductRepository.java
      └── Product.java
```

La arquitectura hexagonal (también conocida como *puertos y adaptadores*) organiza el código para separar la lógica empresarial de las cuestiones externas. Busca un modelo de dominio central rodeado de adaptadores que se conectan con el mundo exterior. Este patrón hace

hincapié en el aislamiento de las reglas empresariales de las implementaciones técnicas. A continuación se muestra un ejemplo de código con una arquitectura hexagonal:

```
src/
  ├── domain/                  // Business logic
  │   ├── model/
  │   └── service/
  ├── application/             // Use cases, orchestration
  │   └── service/
  ├── ports/                   // Interfaces for adapters
  │   ├── input/
  │   └── output/
  └── adapters/                // Technical implementations
      ├── web/
      ├── persistence/
      └── messaging/
```

La arquitectura*de microservicios* divide la funcionalidad en múltiples servicios independientes. Si trabajas en un entorno de microservicios, necesitarás comprender tanto la arquitectura de tu servicio específico como la forma en que encaja en el ecosistema más amplio. En el siguiente ejemplo, cada servicio de nivel superior, como *el servicio de usuario*, es su propio microservicio:

```
microservices-system/
├── user-service/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
```

```
│   ├── models/
│   └── Dockerfile
├── product-service/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
│   ├── models/
│   └── Dockerfile
├── order-service/
│   ├── controllers/
│   ├── services/
│   ├── repositories/
│   ├── models/
│   └── Dockerfile
├── api-gateway/
│   ├── config/
│   ├── filters/
│   └── routes/
└── shared/
    ├── common-models/
    └── utils/
```

Comprender el flujo de ejecución

El flujo de ejecución es la secuencia de instrucciones que sigue un programa durante su ejecución, incluyendo todas las decisiones, bucles y llamadas a funciones que determinan qué código se ejecuta y en qué orden.

Ahora que tienes una visión general del proyecto y comprendes su propósito, puedes empezar a examinar el código. En este momento no

estamos escribiendo ningún código ni realizando ningún análisis en profundidad, solo estamos tratando de comprender cómo fluye el código. En esta sección, aprenderás técnicas para comprender el flujo de ejecución mediante la búsqueda de puntos de entrada de la aplicación, el seguimiento de solicitudes y respuestas, y el aprendizaje de cómo localizar dependencias externas.

Encontrar puntos de entrada a la aplicación

Una aplicación tiene un conjunto de puertas que actúan como puntos de entrada a tu aplicación. Estas puertas se presentan en forma de método principal de la aplicación, API públicas, interfaces de usuario web y mucho más. Estas puertas de entrada a la aplicación son un buen punto de partida si deseas aprender sobre los diferentes flujos de ejecución de una aplicación.

Por ejemplo, en una aplicación web Spring Boot,[1] puedes encontrar la clase principal buscando la anotación `@SpringBootApplication` y localizando el método principal:

```
@SpringBootApplication
public class PetClinicApplication {
    public static void main(String[] args) {
        SpringApplication.run(PetClinicApplication.class, args);
    }
}
```

Una vez que hayas identificado uno de los puntos de entrada de la aplicación, puedes comenzar a seguir el flujo de ejecución del código. Aquí

es donde el depurador de tu IDE puede ser una herramienta realmente valiosa. Establece un punto de interrupción en el método run y luego utiliza la funcionalidad de paso a paso. Esto te dará una idea de lo que el marco está haciendo bajo el capó y te proporcionará un excelente punto de partida para comprender el código base.

Para obtener más información sobre los puntos de entrada de la aplicación, puedes localizar las API públicas, que en esta aplicación de ejemplo PetClinic son los puntos finales REST. La siguiente sección `@GetMapping` nos indica que podemos enviar una solicitud a */pets/new* y que este es el método que se ejecutará. Ahora podemos utilizar las funciones del IDE y nuestras herramientas de depuración para seguir la ruta de ejecución:

```
@GetMapping("/pets/new")
public String initCreationForm(Owner owner, ModelMap model) {
   Pet pet = new Pet();
   owner.addPet(pet);
   return VIEWS_PETS_CREATE_OR_UPDATE_FORM;
}
```

Hay muchos puntos de entrada que pueden ayudarte a trazar el flujo de ejecución de una aplicación. Estos son algunos de los más comunes, pero hay muchos más:

Métodos main/Bootstrap

El punto de partida tradicional de la ejecución (como `main()` de Java).

API/controladores públicos

: Puntos finales que exponen la funcionalidad a sistemas externos

Controladores/receptores de eventos

: Código que se ejecuta en respuesta a eventos o desencadenantes específicos

Tareas/trabajos programados

: Funcionalidad que se ejecuta a intervalos predeterminados

Ganchos de ciclo de vida

: Métodos llamados durante la creación, el inicio o el apagado de componentes

Puntos de conexión/extensión

: Interfaces diseñadas para ampliar la funcionalidad de la aplicación

Consumidores de mensajes

: Código que responde a mensajes de colas o intermediarios de mensajes

Procesadores de argumentos de línea de comandos

: Lógica que gestiona los parámetros de inicio

Desencadenadores de bases de datos/procedimientos almacenados

: Código del lado del servidor que se ejecuta en respuesta a cambios en los datos

La clave es saber cómo encontrar las puertas adecuadas para abrir. Una vez identificadas, puedes utilizarlas como puntos de entrada para comprender el flujo de una aplicación.

Seguir los datos: rastrear los recorridos de las solicitudes

Cuando se trabaja con bases de código existentes, especialmente aplicaciones web y API (REST, GraphQL, gRPC), rastrear el recorrido de una solicitud a través de un sistema es una excelente manera de comprender cómo funciona el código y los sistemas involucrados. Si bien la documentación o las pruebas pueden indicarte lo que se supone que debe suceder, seguir una solicitud revelará lo que realmente sucede.

El seguimiento de solicitudes es valioso porque hace lo siguiente:

- Revela la ruta real de ejecución a través de múltiples componentes y sistemas.
- Ayuda a identificar todas las capas involucradas (controladores, servicios, repositorios, etc.).
- Expone las transformaciones de datos que se producen a lo largo del proceso
- Descubre la lógica empresarial oculta y las reglas de validación
- Muestra cómo se gestionan los errores en la práctica.

Como desarrollador que explora una nueva base de código, tienes a tu disposición algunas herramientas realmente excelentes para rastrear solicitudes. En esta sección, aprenderás sobre algunas de estas herramientas. Es posible que no necesites utilizarlas todas, pero es útil saber cuáles hay disponibles. Prueba algunas de ellas y descubre cuáles te funcionan mejor en tu flujo de trabajo.

Herramientas de desarrollo del navegador

Cuando trabajas c o en aplicaciones web, las herramientas de desarrollo del navegador son un buen punto de partida para inspeccionar una solicitud y una respuesta. En el siguiente ejemplo, estamos ejecutando la aplicación PetClinic y añadiendo una nueva mascota a un propietario (Figura 6-1).

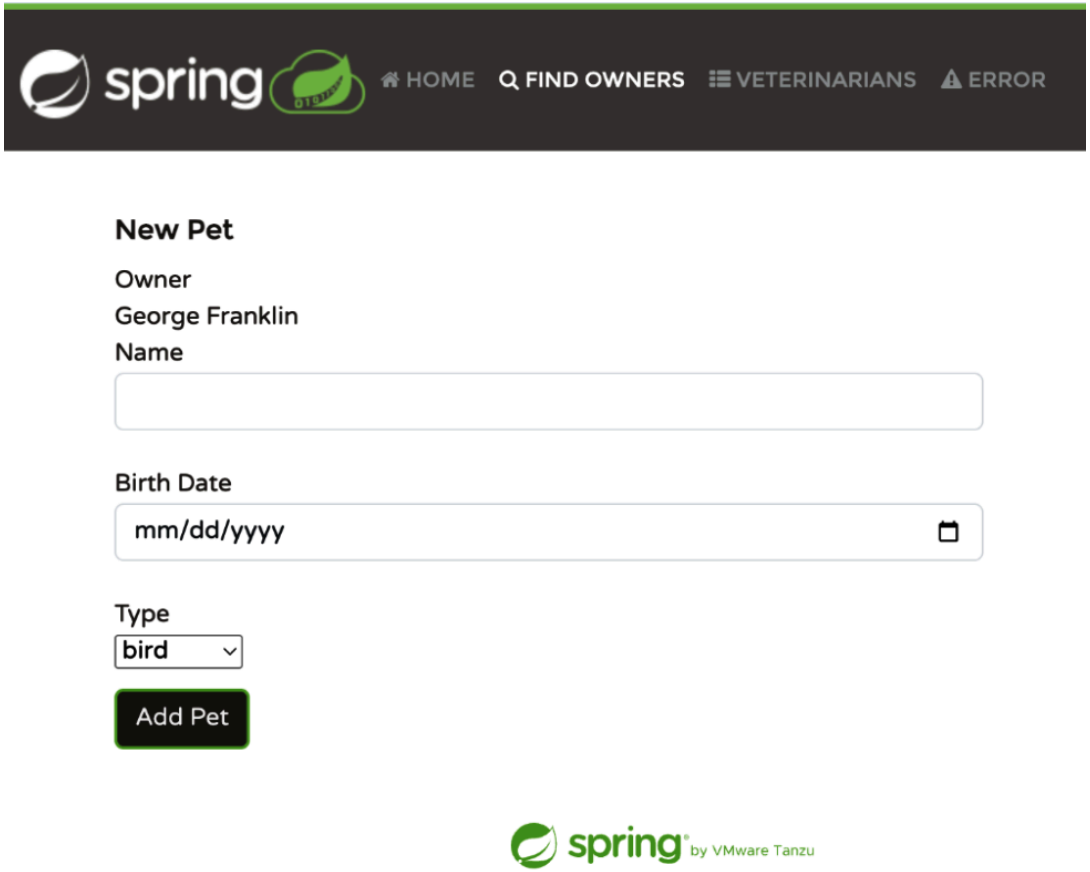


Figura 6-1. La aplicación Spring PetClinic: añadir una nueva mascota al formulario del propietario

Cuando rellenas el formulario y haces clic en el botón Añadir mascota, se envía una solicitud al siguiente punto final:

```
@PostMapping("/pets/new")
public String processCreationForm(Owner owner, @Valid Pet pet,
BindingResult result,
      RedirectAttributes redirectAttributes) {

   if (StringUtils.hasText(pet.getName()) && pet.isNew() &&
   owner.getPet(pet.getName(), true) != null)
      result.rejectValue("name", "duplicate", "already exists");

   LocalDate currentDate = LocalDate.now();
   if (pet.getBirthDate() != null && pet.getBirthDate().isAfter(currentDate)) {
      result.rejectValue("birthDate", "typeMismatch.birthDate");
   }

   if (result.hasErrors()) {
      return VIEWS_PETS_CREATE_OR_UPDATE_FORM;
   }

   owner.addPet(pet);
   this.owners.save(owner);
   redirectAttributes.addFlashAttribute("message", "New Pet has been Added");
   return "redirect:/owners/{ownerId}";
}
```

Si abrís las herramientas de desarrollo e inspeccionás la pestaña Red, verás la solicitud POST al punto final */pets/new*, donde podrás examinar los encabezados, la carga útil, la respuesta y mucho más, como se muestra en la Figura 6-2.

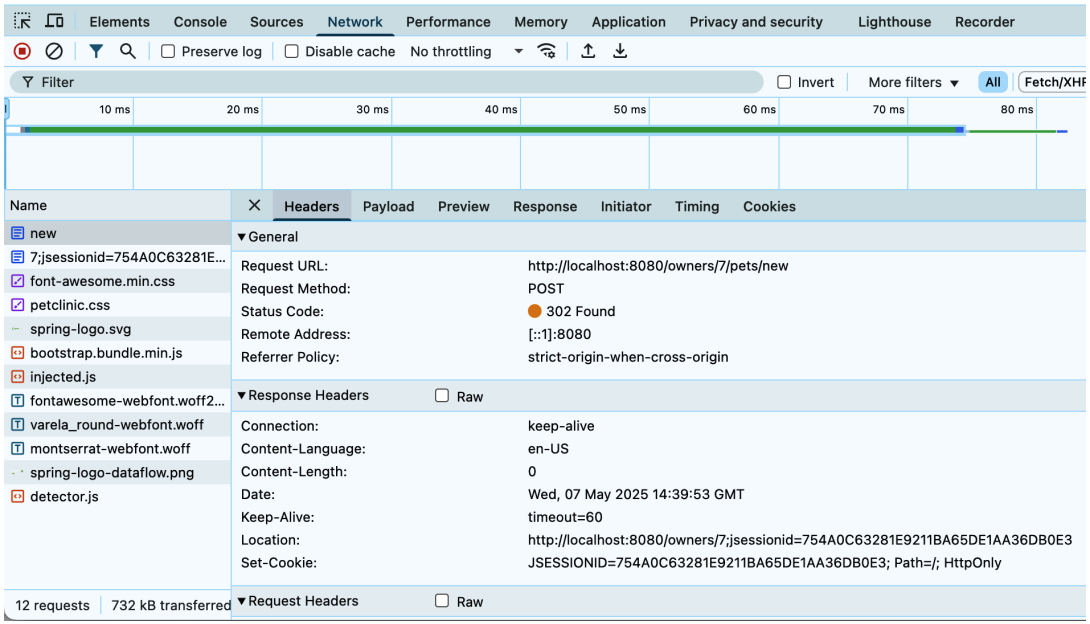


Figura 6-2. Herramientas de desarrollode Chrome que muestran una solicitud POST a /pets/new

Herramientas de prueba de API

Si deseas probar directamente un punto final de la API y se trata de una simple solicitud GET, puedes introducir la URL en tu navegador y ver el resultado. Cuando se trata de cualquier otro método de solicitud como POST/PUT/PATCH/DELETE, necesitarás una herramienta de prueba de API. Hay algunas herramientas realmente excelentes en el mercado, como Postman, Insomnia y Bruno. También encontrarás complementos para muchos de los principales IDE que contienen funciones similares.

En la sección anterior, enviamos el formulario desde el navegador y utilizamos herramientas de desarrollo para inspeccionar la solicitud POST en segundo plano. ¿Qué pasaría si no quisieras pasar por la interfaz de usuario y probar directamente el punto final de la API? Aquí

es donde las herramientas de prueba de API brillan, ya que te permiten enviar una solicitud POST junto con encabezados, autorización, un cuerpo de solicitud y mucho más. Después de enviar la solicitud, puedes inspeccionar la respuesta, lo que te permite omitir por completo la interfaz de usuario (Figura 6-3).

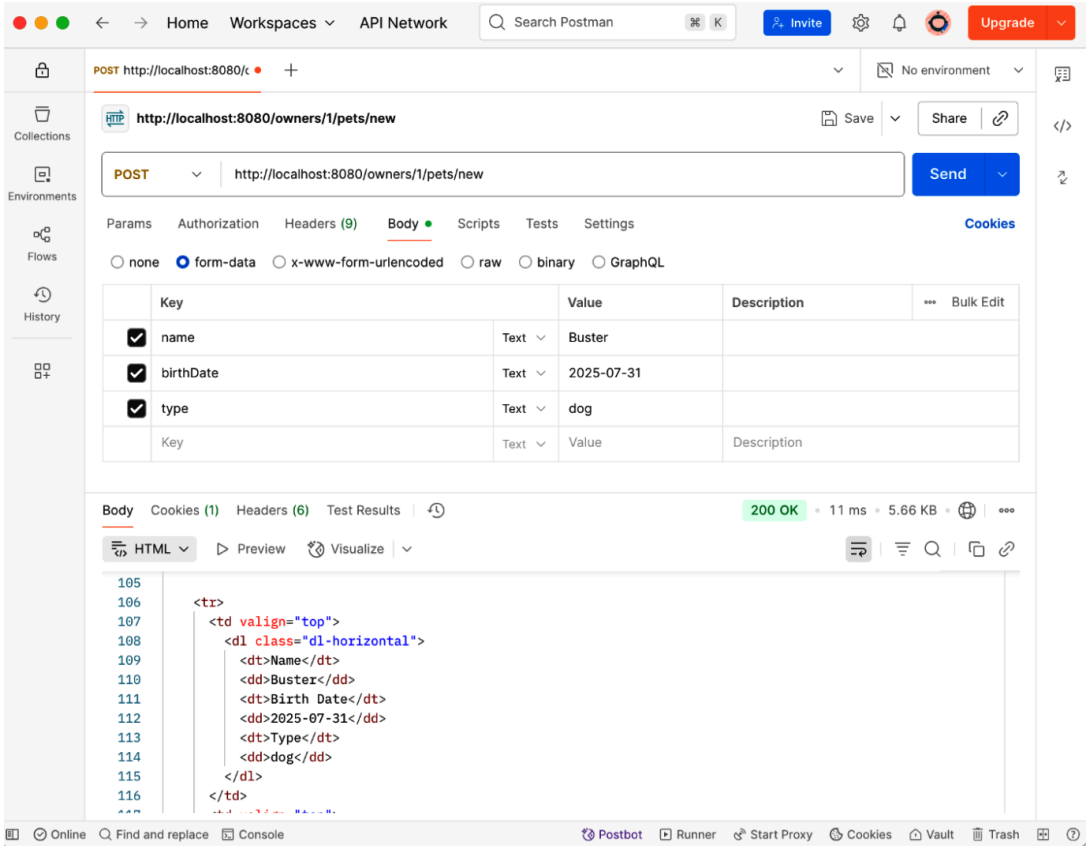


Figura 6-3. Herramienta de prueba de APIPostman enviando una solicitud POST al punto final /posts/new

Puedes probar este punto final con una herramienta como Postman y examinar lo siguiente:

Campos y tipos de datos obligatorios

Identifica qué campos son obligatorios en el cuerpo de la solicitud y qué formato esperan (cadenas, números, matrices, etc.).

Comportamiento de la validación de entradas

Prueba cómo gestiona la API los datos no válidos, los campos que faltan, el JSON malformado y los casos extremos, como cadenas vacías o valores nulos.

Autenticación y autorización

Verifica que los puntos finales protegidos rechacen correctamente las solicitudes no autorizadas y acepten credenciales válidas.

Estructura de respuesta y códigos de estado

Examina qué datos se devuelven en las solicitudes correctas frente a los diferentes tipos de errores (400, 401, 404, 500, etc.).

Calidad de los mensajes de error

Evalúa si las respuestas de error proporcionan información clara y útil para la depuración.

Características de rendimiento

Mide los tiempos de respuesta e identifica cualquier punto final que sea inusualmente lento.

Registros

Los registros proporcionan información sobre el comportamiento interno del sistema. En muchas aplicaciones, los registros pueden ser la única herramienta disponible para depurar problemas o rastrear el re-

corrido de una solicitud en producción. Al examinar el código, busca sentencias de registro como esta:

```
log.info("Finding owner with id: {}", ownerId);
```

El siguiente punto final enumerará todas las mascotas de un propietario en particular. Si realizas una solicitud a este punto final con un ID de propietario válido, verás dos nuevas líneas en el registro (Figura 6-4). También notarás que las llaves se sustituyen por los valores reales:

```
@GetMapping("/owners/{id}/pets")
public List<Pet> findAllPets(@PathVariable("id") int ownerId) {
    log.info("Finding owner with id: {}", ownerId);
    Optional<Owner> owner = ownerRepository.findById(ownerId);
    if (owner.isPresent()) {
        var pets = owner.get().getPets();
        log.info("Found {} pets for owner", pets.size());
        return pets;
    }
    return null;
}
```



Figura 6-4. Consolade IntelliJ IDEA mostrando los mensajes de registro del método `findAllPets`

Depuración

Las funciones de depuración de tu IDE son muy valiosas para recorrer la ejecución del código, especialmente cuando necesitas comprender una lógica empresarial compleja. Volvamos a la validación de la creación de mascotas de antes:

```
@PostMapping("/pets/new")
public String processCreationForm(Owner owner, @Valid Pet pet,
BindingResult result
      RedirectAttributes redirectAttributes) {

   // Set breakpoint here
   if (StringUtils.hasText(pet.getName()) && pet.isNew()
   && owner.getPet(pet.getName(), true) != null)
      result.rejectValue("name", "duplicate", "already exists");

   // Additional validation...
}
```

Sabes que la aplicación rechaza un valor para una nueva mascota, pero ¿por qué? Esta condición tiene varias partes que deben ser ciertas para que se produzca un error de nombre duplicado. Al establecer un punto de interrupción en esta línea, puedes recorrer y examinar lo siguiente (Figura 6-5):

Qué evalúa cada condición

¿ `StringUtils.hasText(pet.getName())` a devuelve verdadero? ¿ `pet.isNew()` a es verdadera?

El estado de los objetos

¿Qué mascotas tiene ya este propietario? ¿Cuál es el nombre exacto que se está comprobando?

Resultados de la llamada al método

¿Qué devuelve realmente `owner.getPet(pet.getName(), true)` ?

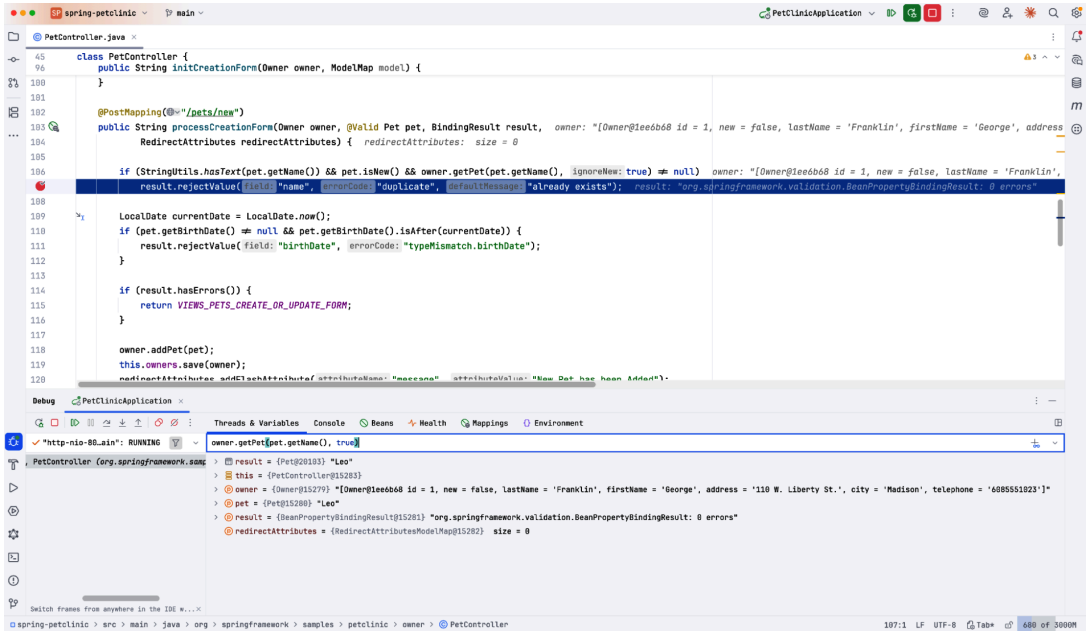


Figura 6-5. Uso del depurador en IntelliJ para recorrer la lógica de negocio

Cuando utilizas herramientas de depuración en tu IDE para recorrer una lógica empresarial compleja, el depurador revela no solo *qué* ruta sigue el código, sino también *por qué* sigue esa ruta. Esto es especialmente valioso cuando el comportamiento no coincide con tus expectativas. Por ejemplo, si los registros indican que un nombre de mascota ya existe, pero estás seguro de que eso es incorrecto, el depurador te permite verificarlo directamente.

Localización de dependencias externas

Rara vez encontrarás una aplicación e e que se ejecute de forma aislada. Las aplicaciones modernas dependen de servicios externos en forma de API públicas y servicios internos con los que se comunican. Una parte importante para comprender el flujo de ejecución es localizar todas y cada una de las dependencias externas con las que se comunica tu aplicación. Estas pueden ser otras API, bases de datos, cachés y mucho más.

Sin una comprensión completa de toda la aplicación, ¿cómo vas a localizar estas dependencias? Un buen punto de partida es examinar los archivos de configuración. En nuestra aplicación PetClinic, creada con Spring Boot, puedes consultar la configuración *de application.properties* o *application.yaml* para ver qué dependencias se han declarado. En el siguiente ejemplo, puedes ver que esta aplicación depende de una base de datos PostgreSQL, un servicio externo y un host de correo electrónico:

```
# database
spring.datasource.url=jdbc:postgresql://db-server:5432/petclinic \
  spring.datasource.driver-class-name=org.postgresql.Driver
# external service
payment.service.endpoint=https://payments.example.com/api/v2
# email host
email.service.host=smtp.company.com
```

Para aplicaciones en contenedores, revisa los archivos Docker Compose o los manifiestos de Kubernetes para identificar los servicios

vinculados:

```
# In docker-compose.yml
services:
  app:
    image: petclinic:latest
    depends_on:
      - mysql
      - redis
      - kafka
```

Localización de marcos y bibliotecas internos

Uno de los mayores retos a la hora de explorar bases de código existentes es encontrar marcos y bibliotecas internos o propios ( ). Se trata de soluciones personalizadas creadas internamente para resolver problemas empresariales específicos o retos técnicos. Aunque en el momento de su creación pudieran ser la decisión correcta, a menudo suponen obstáculos únicos para los nuevos miembros del equipo que intentan explorar y modificar la base de código.

¿Por qué un equipo decidiría crear algo internamente en lugar de optar por una solución externa? Los marcos internos suelen surgir por varias razones:

- El equipo original necesitaba una funcionalidad que no estaba disponible en las bibliotecas existentes.
- Tenían requisitos específicos de seguridad o rendimiento.

- El costo total de incorporar una solución externa era demasiado alto.
- Crearon algo para resolver rápidamente un problema que evolucionó con el tiempo.
- No sabían que existía una solución o no la entendían completamente.

> *El metatrabajo es más interesante que el trabajo real.*
>
> —Neal Ford, director y arquitecto de software en Thoughtworks; autor y conferenciante

Desafortunadamente, estas soluciones suelen adolecer de una documentación deficiente, falta de acceso a los desarrolladores originales y un mantenimiento inconsistente.

Identificación de marcos internos

Los marcos internos pueden ser más difíciles de detectar que las dependencias externas, ya que están integrados directamente en el código base. Estos son los indicadores clave que te permiten saber que estás ante una solución propia:

Patrones de nomenclatura de paquetes

> Busca paquetes con prefijos específicos de la empresa o nombres genéricos como `com.yourcompany.framework`, `utils.common.core` o `internal.shared`.

Clases base personalizadas

Muchas clases que se extienden desde clases base personalizadas con nombres como `BaseService`, `AbstractController` o `CommonEntity` indican código de marco.

Anotaciones personalizadas

Las anotaciones no estándar que no forman parte de los marcos populares suelen indicar soluciones internas:

```
@Entity
@CompanyTable(name = "users", audit = true)
public class User extends BaseCompanyEntity {

    @CompanyField(encrypted = true)
    private String email;

}
```

Clases de utilidad con amplio alcance

Las clases denominadas `AppUtils`, `BusinessHelper` o `SystemManager` suelen gestionar diversas responsabilidades en toda la aplicación.

Configuración pesada

Archivos YAML/XML personalizados de extensión o clases de configuración que no coinciden con los patrones de la biblioteca estándar.

Trabajar con marcos internos

Los siguientes consejos te ayudarán a explorar los marcos internos y a dejar el código en mejores condiciones de las que lo encontraste:

Busca primero ejemplos de uso

Busca en el código base cómo otros desarrolladores utilizan el marco. Los ejemplos reales suelen ser más instructivos que la documentación.

Busca pruebas

El código del marco a veces tiene una mejor cobertura de pruebas, ya que necesita funcionar en múltiples escenarios. Las pruebas revelan el comportamiento previsto y los casos de perímetro.

Documenta a medida que aprendes

Crea notas sobre lo que hace el marco y cómo utilizarlo. Incluso la documentación básica ayuda a los futuros desarrolladores.

Revisa el historial de control de versiones

Utiliza `git blame` y los mensajes de confirmación para comprender por qué se creó el marco y cómo ha evolucionado.

Evalúa las ventajas y desventajas

Considera si el marco sigue aportando valor o se está convirtiendo en una carga de mantenimiento. A veces, sustituir las soluciones internas por bibliotecas estándar es la opción correcta a largo plazo.

Realiza mejoras incrementales

Aplica la regla scout (más adelante se ofrece más información al respecto en este capítulo) mejorando los nombres, añadiendo documentación o ampliando la cobertura de las pruebas a medida que trabajas con el marco.

Construye modelos mentales de forma incremental

Un *modelo mental* es tu representación interna de cómo funciona el sistema, y desarrollar un modelo sólido lleva tiempo y práctica deliberada. Si vas a construir un modelo mental de un sistema, es mucho más fácil hacerlo si desglosas el sistema de forma incremental. En lugar de intentar construir un modelo mental de toda una aplicación de comercio electrónico, toma una sola ruta, como el proceso de pago, y construye a partir de ahí. También es útil evitar que las herramientas o el perfeccionismo se interpongan en el camino. A menudo, basta con un simple modelo dibujado a mano en un bloc de notas.

En esta sección, aprenderás algunos métodos prácticos para construir modelos mentales, como dividir sistemas complejos en otros más pequeños, visualizar esos modelos y ampliar gradualmente tus conocimientos.

NOTA SOBRE IA

Otra ventaja e e en estos días de diagramas basados en texto es que la IA generativa puede ayudarte con un borrador inicial que puedes perfeccionar manualmente. Puedes describir tu sistema o proceso a una herramienta de IA y pedirle que genere sintaxis Mermaid, y luego iterar sobre el resultado para que se adapte a tus necesidades específicas.

Desglosar sistemas complejos

Desglosar sistemas complejos es una parte importante de la creación de modelos mentales. Cuando te unes a un nuevo proyecto con un código base grande, *intentar comprenderlo todo de una vez es abrumador e ineficaz*. En su lugar, debes centrarte en desglosar los sistemas complejos en partes manejables.

En la sección anterior aprendiste sobre los puntos de entrada de las aplicaciones, y este es un buen punto de partida. Por ejemplo, en una aplicación de comercio electrónico, es posible que hayas definido algunos de estos flujos de ejecución:

Nuevo flujo de clientes

: Un nuevo usuario se registra, se convierte en cliente y recibe un cupón promocional.

Flujo de pago

: Qué ocurre cuando un usuario completa una compra con productos en su carrito.

Flujo de reseñas de productos

Un cliente proporciona comentarios sobre un producto comprado.

Estos son excelentes ejemplos de cómo dividir un sistema complejo en partes más pequeñas y manejables que puedas comprender. Una vez que hayas visto cómo se registra una nueva reseña de producto y hayas visto el código asociado a esos pasos, puedes empezar a construir un modelo mental de este proceso. Cada persona aprende de manera diferente, y tal vez tú seas de los que quieren construir un modelo mental en torno al código y no al proceso. Si tu aplicación utiliza una disposición por paquetes según las funciones, podrías construir modelos de cada función. En el siguiente ejemplo, podemos ver todo el código asociado a la función *del carrito*:

```
src/
├── cart/
│   ├── CartController.java
│   ├── CartService.java
│   ├── CartRepository.java
│   ├── Cart.java
│   ├── CartItem.java
│   └── dto/
│       ├── CartDTO.java
│       └── CartItemDTO.java
```

Visualización de tus modelos mentales

Visualizar las relaciones entre el código puede ayudarte a crear estos modelos mentales. En el capítulo 4, aprendiste sobre el modelado de

software y algunos de los tipos de diagramas que tienes a tu disposición. Tus visualizaciones no tienen por qué ser profesionales: pueden ir desde simples bocetos hasta diagramas UML completos. Utiliza el nivel de detalle que te ayude a comprender el sistema. La primera parte de la visualización de tu modelo mental es elegir el tipo de visualización. Los diferentes tipos de visualizaciones tienen diferentes propósitos en la ingeniería de software :

Diagramas de flujo

Ayudarte a comprender los procesos secuenciales y los puntos de decisión

Diagramas de entidad-relación

Aclarar las estructuras de datos y sus relaciones

Diagramas de secuencia

Ilustrar cómo interactúan los componentes a lo largo del tiempo

Diagramas de componentes

Mostrar la arquitectura de alto nivel y las dependencias

Mapas mentales

Te ayudarán a organizar conceptos relacionados de forma jerárquica.

Puedes empezar con una representación sencilla y añadir detalles a medida que aumentes tu comprensión del sistema. Por ejemplo, al vi-

sualizar el flujo de pago en un sistema de comercio electrónico, puedes empezar solo con los componentes principales. A continuación se muestra un sencillo diagrama de flujo escrito en sintaxis Mermaid, una popular herramienta de diagramación basada en Markdown:

```
graph TD
    A[Shopping Cart] --> B[Checkout Form]
    B --> C[Payment Processing]
    C --> D[Order Confirmation]
```

El código anterior puede generar un diagrama de flujo como el que se muestra en la Figura 6-6.

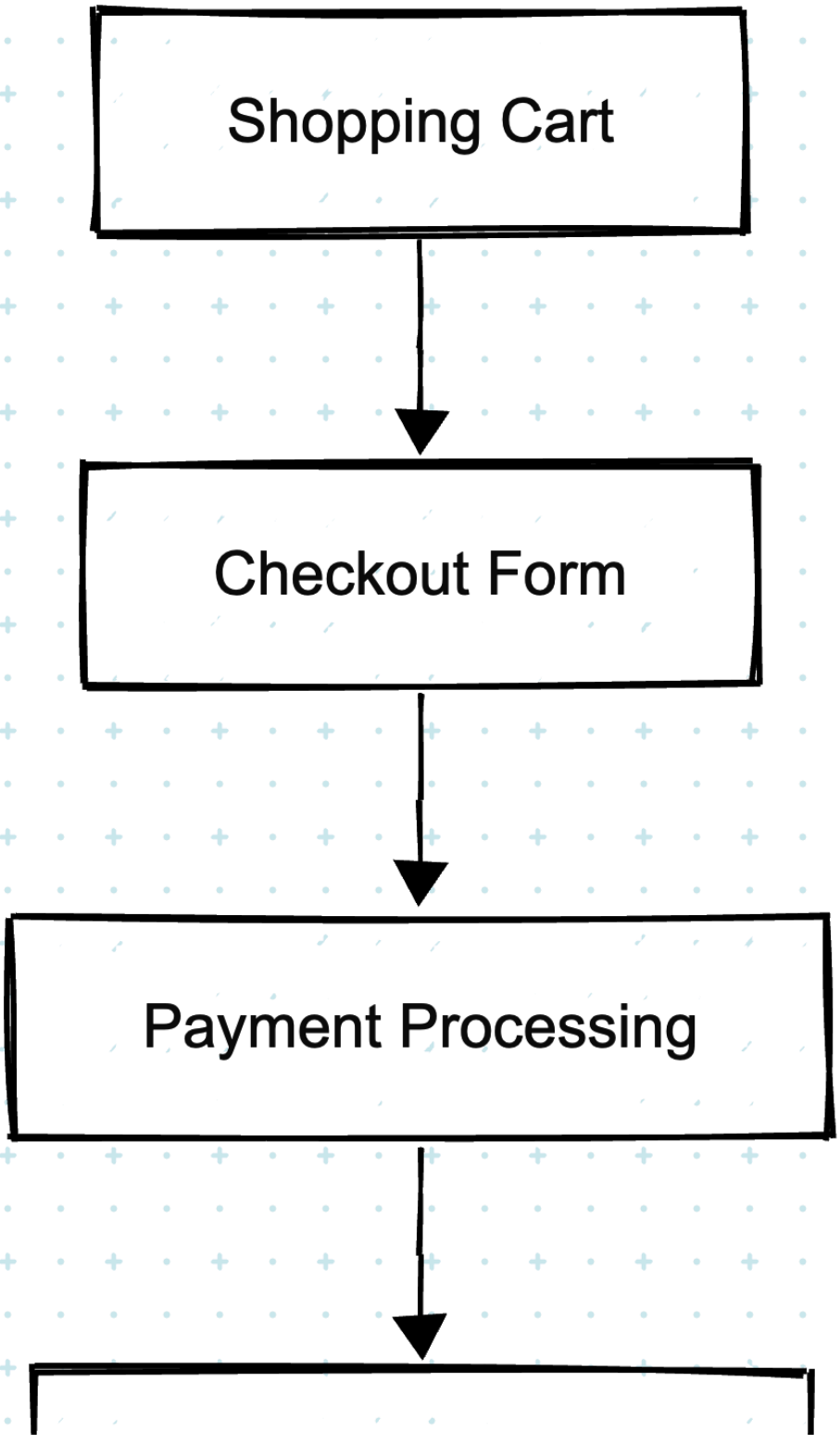

Shopping Cart
Checkout Form
Payment Processing



Figura 6-6. Diagrama de flujo de un proceso de pago de muestra en una aplicación de comercio electrónico

A medida que aprendas más, puedes ampliar este diagrama de flujo para incluir servicios específicos, interacciones con bases de datos e integraciones externas. Por ejemplo:

```
sequenceDiagram
    participant U as User
    participant FE as Frontend
    participant API as API Layer
    participant CS as CartService
    participant PS as PaymentService
    participant OS as OrderService
    participant DB as Database
    participant PP as Payment Provider

    U->>FE: Click "Checkout"
    FE->>API: POST /checkout
    API->>CS: validateCart()
    CS->>DB: getCartItems()
    DB-->>CS: cartItems
    CS-->>API: validation result
    API->>PS: processPayment()
    PS->>PP: authorizeCharge()
    PP-->>PS: authorization
    PS-->>API: payment result
```

```
API->>OS: createOrder()
OS->>DB: saveOrder()
DB-->>OS: orderConfirmation
OS-->>API: order details
API-->>FE: success response
FE-->>U: Display confirmation
```

Este código genera el diagrama de flujo que se muestra en la Figura 6-7.

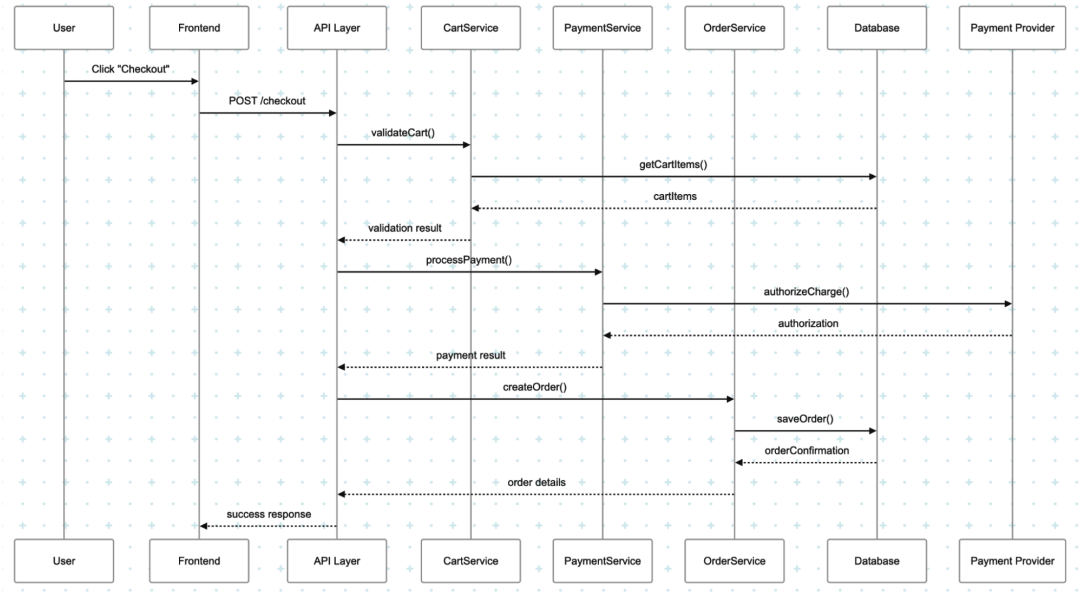


Figura 6-7. Un diagrama de flujo más grande para una aplicación de comercio electrónico

Amplía gradualmente tu alcance

Una vez que comprendas los componentes individuales, amplía gradualmente tu enfoque al comportamiento del sistema en su conjunto:

1. Comienza con métodos individuales de forma aislada.

2. Pasa a las clases y sus dependencias directas.
3. Continúa con flujos a nivel de características que abarquen múltiples componentes.
4. Por último, comprende cuestiones transversales como la seguridad o la gestión de transacciones.

Este enfoque incremental evita la sobrecarga cognitiva al tiempo que construye una visión global del sistema.

Al invertir tiempo en crear modelos mentales precisos, tomarás mejores decisiones al modificar el código base. Recuerda que se trata de un proceso continuo que lleva tiempo. Incluso los miembros más experimentados del equipo siguen perfeccionando su comprensión del sistema en su conjunto a medida que este evoluciona. Ahora que has aprendido a comprender códigos base desconocidos, veamos un proceso de ejemplo para trabajar con código existente.

Un proceso de ejemplo

Con el tiempo, desarrollarás una sensibilidad para trabajar con el código existente. Sin embargo, puede ser útil contar con un proceso. Christopher Judd enseña el siguiente método en sus campamentos de entrenamiento:

1. Clona el proyecto desde el sistema de gestión del código fuente.
2. Revisa el archivo README, los estándares de codificación, la arquitectura y otra documentación de apoyo.

3. Toma notas (arquitectura, comandos de compilación, SQL común, estándares de la industria, términos, etc.) a medida que avanzas. ¡No tengas miedo de compartir estas notas con tus compañeros de equipo!
4. Revisa los scripts de compilación.
5. Revisa las dependencias del proyecto.
6. Revisa la estructura del proyecto (paquetes, espacios de nombres, módulos, artefactos, etc.).
7. Revisa los procesos de CI/CD.
8. Instala las dependencias del proyecto (herramientas de compilación, entornos de ejecución, lenguajes).
9. Desde tu IDE:
    a. Ejecuta y depura la aplicación.
    b. Añade puntos de interrupción a los métodos interesantes, conectándolos de nuevo a la aplicación en ejecución.
    c. Ejecuta y depura las pruebas unitarias.
    d. Añade puntos de interrupción a los métodos interesantes, conectándolos de nuevo a la aplicación en ejecución.
10. Desde la línea de comandos:
    a. Compila los artefactos del proyecto.
    b. Ejecuta las pruebas unitarias.
    c. Inicia los contenedores necesarios (como el almacén de datos).
    d. Ejecuta la aplicación localmente (puertos, URL, etc.).

Recuerda que comprender un código base existente es un proceso iterativo. No podrás comprenderlo todo a la primera, y eso es completamente normal. Cada vez que trabajes en un problema o en una nueva función, adquirirás un conocimiento más profundo.

Realizar cambios de forma segura

Ahora que tienes algunos consejos y un proceso para comprender un código base existente, es hora de aprender a modificarlo de forma segura. Es probable que los sistemas existentes hayan pasado por numerosas iteraciones, funciones y mucho más. No puedes simplemente subir al ring como un boxeador salvaje lanzando golpes y gastando toda tu energía en el primer asalto. Debes navegar por el código base de forma metódica y tener cuidado de no romper nada. Probablemente hayas oído la frase «muévete rápido y rompe cosas», y puede que tenga sentido en ciertos escenarios, pero este no es uno de ellos.

Refactorización segura

Cuando se trabaja con sistemas heredados ( ) y bases de código existentes, a menudo existe la posibilidad de refactorizar el código para mejorar la legibilidad, el rendimiento y otros aspectos. En esta situación, es importante asegurarse de tener un plan y no intentar hacer demasiado de una sola vez.

Confía en las pruebas existentes

Como aprendiste en la sección anterior, las pruebas desempeñan un papel importante en la comprensión del código base, pero también hacen que tu trabajo de refactorización del código sea más seguro y fácil. Si el proyecto cuenta con un conjunto completo de pruebas, te conviene apoyarte en ellas en gran medida. Las pruebas sirven como red de seguridad, ya que te permiten realizar pequeños cambios, ejecutar las

pruebas y asegurarte de que no has afectado a ninguna otra parte del proyecto.

En el siguiente ejemplo, estás refactorizando el método `calculateTotal` para separar el cálculo del subtotal y la aplicación de un descuento. Al refactorizar, debes evitar modificar las pruebas, ya que el objetivo es mejorar la estructura del código conservando su comportamiento. Después de cada paso de esta refactorización, debes ejecutar las pruebas asociadas a este método para asegurarte de que el comportamiento no ha cambiado:

```
// Before refactoring
public double calculateTotal(Order order) {
    double total = 0;
    for (OrderItem item : order.getItems()) {
        total += item.getPrice() * item.getQuantity();
    }
    if (order.hasDiscount()) {
        total = total * (1 - order.getDiscountRate());
    }
    return total;
}

// After refactoring
public double calculateTotal(Order order) {
    double subtotal = calculateSubtotal(order);
    return applyDiscount(subtotal, order);
}

private double calculateSubtotal(Order order) {
    return order.getItems().stream()
```

```
            .mapToDouble(item -> item.getPrice() * item.getQuantity())
            .sum();
    }

    private double applyDiscount(double amount, Order order) {
        return order.hasDiscount()
            ? amount * (1 - order.getDiscountRate())
            : amount;
    }
```

Añade pruebas antes de refactorizar

En el ejemplo anterior, nos basamos en las pruebas para asegurarnos de que nada más se viera afectado en el ámbito más amplio del proyecto. Si la aplicación no tiene una cobertura de pruebas adecuada para los cambios que propones, esta es tu oportunidad de añadir pruebas antes de refactorizar cualquier código existente. Este enfoque, a veces denominado*refactorización basada en pruebas ( )*, sigue estos pasos:

1. Escribe pruebas que documenten el comportamiento actual.
2. Verifica que las pruebas se superen con la implementación actual.
3. Refactoriza el código.
4. Verifica que las pruebas sigan superándose con la nueva implementación.

La siguiente prueba unitaria verifica que el servicio de pedidos aplica correctamente un descuento del 10 % al calcular el total de un pedido que cumple los requisitos para el descuento:

```
@Test
public void calculateTotalAppliesDiscountWhenOrderHasDiscount() {
    // Given
    OrderItem item1 = new OrderItem("Product1", 10.0, 2);
    OrderItem item2 = new OrderItem("Product2", 15.0, 1);
    Order order = new Order(Arrays.asList(item1, item2), true, 0.1);

    // When
    double total = orderService.calculateTotal(order);

    // Then
    assertEquals(31.5, total, 0.001); // (10*2 + 15*1) * 0.9 = 31.5
}
```

Añadir pruebas como esta antes de realizar cambios te ayuda a comprender el comportamiento existente y te da la confianza de que tu refactorización conserva ese comportamiento.

CONSEJO

Es importante recordar que, al añadir pruebas a bases de código existentes, tu primer objetivo es documentar el comportamiento actual, no cambiarlo. Aunque conozcas una forma mejor de refactorizar el código o corregir algo que estás seguro de que es un error, primero debes escribir las pruebas para confirmar el comportamiento actual. A continuación, puedes volver y corregir los errores o realizar refactorizaciones adicionales una vez que las pruebas estén en marcha.

La regla scout

La regla scout del es un principio del desarrollo de software que establece: «Deja siempre el código mejor de lo que lo encontraste». Se inspira en la filosofía de acampada de los Boy Scouts of America, donde se enseña a los scouts a dejar el campamento más limpio de lo que lo encontraron.

La regla scout anima a realizar pequeñas mejoras incrementales en la calidad del código cada vez que tocas un archivo. Estas mejoras pueden incluir lo siguiente:

- Añadir la documentación que falta
- Mejorar los nombres de variables o métodos para mayor claridad
- Desglosar métodos demasiado complejos
- Eliminar código muerto
- Corregir errores menores que descubras

En el siguiente ejemplo, simplemente mejorando el nombre del método, añadiendo parámetros de tipo, utilizando un bucle « `for` » mejorado y añadiendo documentación, has conseguido que el código sea mucho más fácil de mantener sin cambiar su funcionalidad básica:

```
// Before applying the Scout Rule
public void prcs(List l) {
    for (int i = 0; i < l.size(); i++) {
        Object o = l.get(i);
        // Process object
        // ...
```

```
    }
}

// After applying the Scout Rule
/**
 * Processes a list of customer records and updates their status in the database.
 *
 * @param customers The list of customers to process
 */
public void processCustomers(List<Customer> customers) {
    for (Customer customer : customers) {
        // Process customer
        // ...
    }
}
```

Aunque la regla scout te anima a limpiar el código sobre la marcha, debes encontrar un equilibrio entre la limpieza y la tarea que tienes entre manos. En bases de código más grandes, podrías pasar todo tu tiempo simplemente mejorando el código, solo para darte cuenta de que no has arreglado lo que te propusiste arreglar originalmente.

No todo el código que se puede mejorar debe mejorarse inmediatamente. Ten en cuenta estos factores a la hora de decidir si refactorizar.

Puedes refactorizar el código en estas circunstancias:

- El código en el que estás trabajando es difícil de entender.
- Necesitas añadir una función a esa zona.
- Estás corrigiendo un error en el código.

- El código está causando problemas de rendimiento.
- Varios desarrolladores trabajan con frecuencia en esta área.

Debes considerar posponer la refactorización en estos casos:

- El código funciona bien y rara vez necesita cambios.
- Tienes una fecha límite muy ajustada.
- El riesgo de romper el código supera los beneficios.
- La refactorización requeriría cambios de extensión que van más allá de tu tarea actual.
- No tienes una cobertura de pruebas adecuada para garantizar la seguridad.

Cambios pequeños y reversibles

Realizar cambios incrementales y fácilmente reversibles e es es la forma más segura de modificar el código existente sin afectar a todo el sistema. Esta es una habilidad que debes practicar continuamente para dominarla. Aunque puedas sentir la tentación de abordar problemas complejos de una sola vez, y los propietarios de los productos puedan presionar para que se realicen cambios más grandes y con mayor impacto, tu objetivo debe ser realizar modificaciones más pequeñas que no perturben el sistema en su conjunto.

Estrategias de gestión del cambio

Una estrategia de gestión del cambio para equipos de software es un enfoque estructurado para gestionar las modificaciones en tus aplicaciones. Se trata, esencialmente, de un sistema para proponer, revisar,

implementar y realizar un seguimiento de los cambios con el fin de mantener la calidad del código y la alineación del equipo. Cuando se trata de cambios pequeños y reversibles, tu estrategia de gestión del cambio debe dar prioridad a la seguridad y la visibilidad.

A continuación, se presentan algunas estrategias prácticas que puedes utilizar en tu enfoque de gestión del cambio:

Haz que los cambios sean visibles

Asegúrate de que tus cambios estén bien documentados y sean fáciles de entender para los demás. No es necesario que sean exhaustivos, pero deben explicar qué motivó el cambio y cómo lo implementaste.

Incorpora la verificación

Incluye formas de verificar que tus cambios funcionan según lo esperado. Los métodos de verificación para los equipos pueden incluir pruebas automatizadas, revisiones de código y análisis estáticos.

Planifica la reversión

La ley de Murphy establece que todo lo que puede salir mal, saldrá mal. Ten siempre un plan para revertir los cambios si surgen problemas. Cuando realizas cambios más pequeños, como los que proponemos aquí, revertirlos debería ser más fácil.

Monitorea los efectos

Estate atento a las consecuencias inesperadas de tus cambios. Aquí es donde realmente vale la pena tener una buena observabilidad de tus aplicaciones.

Incrementos pequeños y comprobables

Una vez que te acostumbras a trabajar con incrementos más pequeños que puedes probar, tu trabajo de modificar los códigos base existentes se vuelve menos estresante y más fácil de gestionar. Cuando adoptas este enfoque, obtienes las siguientes ventajas:

- Reduce el riesgo al limitar el alcance de cada cambio.
- Hace que las pruebas sean más específicas y eficaces.
- Hace que las revisiones de código sean más manejables.
- Permite resolver más fácilmente los problemas que puedan surgir.

Esto puede significar rechazar solicitudes de funciones si son demasiado grandes y dividirlas en funciones más pequeñas para controlar el tamaño del cambio. Recuerda que es mejor realizar varios cambios pequeños y exitosos que un solo cambio grande que introduzca errores.

Supongamos que recibes una solicitud para modificar la forma en que tu sistema procesa un pedido. En el siguiente ejemplo, puedes ver que el proceso de pedido ya hace demasiado y es candidato para una refactorización. En lugar de intentar cambiar la lógica de todo el método, considera dividir algunas de las funcionalidades en métodos más pequeños, escribir pruebas para esos métodos y validar que cada uno funciona correctamente antes de pasar a otro:

```
// Instead of changing this entire method at once...
public void processOrder(Order order) {
    // Validate order
    // Calculate totals
```

```
    // Apply discounts
    // Calculate shipping
    // Apply taxes
    // Process payment
    // Update inventory
    // Send confirmation
}

// Break it down into smaller, separate changes:
// 1. First, extract the validation logic
private boolean validateOrder(Order order) {
    // Validation logic here
    return isValid;
}

// 2. Next, extract the totals calculation
private double calculateOrderTotal(Order order) {
    // Total calculation logic here
    return total;
}

// 3. Continue with other extractions, testing after each change
```

Buenas prácticas para el control de versiones

Probablemente, la mayoría de los nuevos proyectos en los que trabajas ( ) ya cuentan con un control de versiones. En algunos casos, es posible que te encuentres trabajando con código existente que carece de un control de versiones adecuado o que necesites establecer las buenas prácticas desde el principio:

Bases de código heredadas

Es posible que te encuentres con proyectos antiguos que tienen control de versiones pero carecen de buenas prácticas. Estas aplicaciones pueden enviar todo al main sin una estrategia de ramificación. Pueden contener mensajes de envío inconsistentes, cambios masivos sin fusionar y repositorios con archivos generados o dependencias que nunca deberían haberse rastreado en primer lugar.

Proyectos personales o a pequeña escala

Al iniciar tus propios proyectos o trabajar en equipos muy pequeños, es tentador omitir el control de versiones. Puedes pensar «solo soy yo» o «podemos comunicarnos directamente», pero establecer buenos hábitos desde el principio evita la deuda técnica y facilita las cosas cuando el proyecto crece o cuando necesitas incorporar a otras personas.

Contribuciones de código abierto

Al bifurcar repositorios o contribuir a proyectos de código abierto, debes comprender y seguir las convenciones de control de versiones del proyecto, al tiempo que mantienes tu propia bifurcación de forma adecuada.

Trabajo como contratista o consultor

Al unirte a proyectos existentes, es posible que tengas que establecer o mejorar prácticas de control de versiones que antes no eran prioritarias.

Independientemente del escenario en el que te encuentres, los sistemas de control de versiones como Git son esenciales para realizar cambios seguros en los códigos base existentes. Cuando se utilizan de for-

ma eficaz, proporcionan redes de seguridad que te permiten experimentar, realizar un seguimiento del historial y colaborar sin temor a perder el trabajo o romper la funcionalidad.

Realiza compromisos estratégicos

Cuando trabajes con código existente, tus compromisos deben contar la historia de cómo ha evolucionado el código:

Realiza confirmaciones atómicas

Cada confirmación debe representar un único cambio lógico que pueda funcionar por sí solo. Esto facilita la comprensión, la revisión y, si es necesario, la reversión de los cambios.

Escribe mensajes de confirmación significativos

Un buen mensaje de confirmación explica tanto qué ha cambiado como *por qué*. Los futuros desarrolladores (incluido tú mismo en el futuro) te agradecerán el contexto. El siguiente código muestra ejemplos de mensajes de confirmación buenos y malos:

```
# Poor commit message
Fix bug

# Better commit message
Fix order calculation when a discount code is applied twice

The system was double-applying discount codes when customers
entered them in both mobile and desktop sessions. This fix
ensures the discount is only applied once per order.
```

Realiza confirmaciones con frecuencia

No esperes hasta haber realizado docenas de cambios antes de realizar una confirmación. Las confirmaciones lógicas pequeñas y frecuentes facilitan la identificación de cuándo y dónde se introdujeron los problemas.

Ramifica con prudencia

Las estrategias de ramificación variarán según la organización, el equipo y el proyecto, pero se aplicarán ciertos principios cuando se trabaje con código existente:

Crea ramas de características

Aísla tus cambios en una rama dedicada hasta que estén listos para su integración. Esto mantiene la rama principal estable y te permite experimentar libremente.

Mantén las ramas con una vida corta

Las ramas de larga duración se alejan más del código base principal con el tiempo, lo que dificulta la integración. Intenta fusionar tus cambios en cuestión de días, no de semanas.

Rebasa antes de fusionar

Cuando sea apropiado, rebasa tus cambios sobre la última rama principal para asegurarte de que estás trabajando con la versión más actualizada del código base. *Importante*: el rebase reescribe el historial de confirmaciones, así que úsalo con cuidado en las ramas compartidas.

Solicitudes de incorporación de cambios y revisiones de código

Una solicitud de incorporación de cambios (PR) es una propuesta de un desarrollador para fusionar los cambios de una rama a otra, abriendo el trabajo para su revisión, discusión y comprobaciones automatizadas antes de la integración. Es recomendable que tus PR sean específicas y manejables. Una PR que cambia 1000 líneas de código en 20 archivos es difícil de revisar. En su lugar, intenta crear PR más pequeñas y específicas que aborden una sola cuestión.

En la revisión del código, asegúrate de explicar qué problema estás resolviendo y cómo tus cambios lo abordan. Incluye cualquier referencia que tengas a cuestiones o requisitos relevantes. Las revisiones de código tienen como objetivo mejorar la solución, no criticar al desarrollador. Mantén una actitud abierta a las sugerencias y dispuesta a iterar en tu enfoque.

Como puedes ver, aunque no debes lanzarte a modificar los códigos base existentes con total libertad, no hay por qué tener miedo a dejar huella. Refactoriza de forma segura con pruebas, sigue la regla scout y céntrate en cambios pequeños y reversibles.

Conclusión

Trabajar con código existente es una de esas habilidades fundamentales que pueden llevarte de programador a ingeniería de software. A lo largo de este capítulo, hemos explorado estrategias para navegar, comprender y modificar de forma segura bases de código. Has aprendido a

comprender bases de código desconocidas empezando por el panorama general, revisando la documentación y comprendiendo la arquitectura del proyecto. Hemos visto cómo seguir los flujos de ejecución encontrando puntos de entrada, rastreando los recorridos de las solicitudes e identificando las dependencias externas. Ahora sabes cómo construir modelos mentales de forma incremental desglosando sistemas complejos y visualizando relaciones. También hemos examinado cómo realizar cambios de forma segura mediante una refactorización cuidadosa, siguiendo la regla scout y utilizando las buenas prácticas de control de versiones.

Estas habilidades no solo son útiles, sino que son esenciales para tu crecimiento como ingeniería de software. Aunque crear nuevas aplicaciones desde cero puede ser divertido y emocionante, la realidad es que la mayor parte de tu carrera profesional consistirá en trabajar con código existente. Dominar estas técnicas te ayudará a contribuir de forma rápida y segura a cualquier código base, independientemente de su tamaño o complejidad.

Ponerlo en práctica

Para poner en práctica lo que has aprendido se requiere acción, no solo conocimientos. Los siguientes pasos prácticos te ayudarán a aplicar estas habilidades de navegación y modificación de código cuando trabajes con código existente:

1. Busca un proyecto de código abierto que te interese y dedica 30 minutos a explorar su código base sin escribir nada. Céntrate en com-

prender su estructura y organización.

2. Practica el seguimiento de solicitudes de principio a fin seleccionando una aplicación web (como el ejemplo de PetClinic) y siguiendo el recorrido completo del usuario, desde la interacción con la interfaz de usuario hasta las llamadas a la API y los cambios en la base de datos, utilizando las herramientas de desarrollo del navegador, el registro y la depuración en secuencia.
3. Elige un método de un proyecto de código abierto que sea difícil de entender y refactorízalo para mejorar la legibilidad sin cambiar su comportamiento.
4. Busca un proyecto heredado con una cobertura de pruebas mínima y practica añadiendo pruebas que documenten el comportamiento existente.
5. Crea un diagrama de modelo mental eligiendo un proceso empresarial de una base de código real y crea una representación visual (dibujo a mano, diagrama de flujo, diagrama de secuencia o diagrama de componentes) que muestre cómo fluye realmente el código, y luego valídalo con un miembro del equipo.
6. Realiza un cambio complejo en una base de código existente utilizando pequeñas confirmaciones reversibles. Obtén comentarios de un compañero sobre tu estrategia de confirmación.
7. Aplica la regla scout a un proyecto en el que estés trabajando identificando tres pequeñas mejoras que puedas realizar para dejar el código mejor de lo que lo encontraste.
8. Practica cambios seguros en las dependencias identificando una dependencia externa en un proyecto e investigando cómo actualizarla o sustituirla de forma segura, incluyendo el análisis del impacto y la planificación de la reversión.

Recursos adicionales

- *Refactoring: Improving the Design of Existing Code*, de Martin Fowler (Addison-Wesley Professional, 2018)
- *Working Effectively with Legacy Code*, de Michael Feathers (Prentice Hall, 2004)
- *Getting to Know IntelliJ IDEA*, de Trisha Gee y Helen Scott (JetBrains, 2021)

Utilizaremos la aplicación Spring PetClinic como ejemplo a lo largo de este capítulo. Aunque no seas un experto en Java o Spring, debería resultarte bastante sencillo de entender.